

ML Loop Code-Along - Step 08

Classification: predicting a category

Incremental prerequisite prep for Imperial ML/AI coursework

Prepared for Anthony Watts - June 2026

Rule: nothing appears before it is explained.

8. Classification: predicting a category

Regression predicts numbers. Classification predicts classes. The workflow still uses X, y, split, fit, predict, and evaluate, but the metrics change.

Step 8.1: Import a built-in classification dataset

Why this cell exists

load_breast_cancer is bundled with scikit-learn. It is a convenient numeric classification dataset. Again, this is for learning workflow, not medical advice.

```
from sklearn.datasets import load_breast_cancer
```

What to look for

This gives access to the loader function, not the data itself yet.

Step 8.2: Load classification data

Why this cell exists

The same X/y pattern appears. X contains numeric features. y contains class labels.

```
cancer = load_breast_cancer(as_frame=True)

X_cls = cancer.data
y_cls = cancer.target

print("X_cls:", X_cls.shape)
print("y_cls:", y_cls.shape)
print("Class names:", list(cancer.target_names))
```

What to look for

The target values are encoded as numbers, but they represent categories.

Step 8.3: Check class balance before modelling

Why this cell exists

Accuracy can be misleading when one class is much more common than another. Always look at target counts.

```
class_counts = y_cls.value_counts().sort_index()
class_percentages = y_cls.value_counts(normalize=True).sort_index() * 100

pd.DataFrame({
    "class_name": cancer.target_names,
    "count": class_counts,
    "percentage": class_percentages.round(1),
})
```

What to look for

If a class dominates the dataset, a stupid model can look good by guessing the majority class.

Step 8.4: Split classification data with stratify**Why this cell exists**

stratify=y_cls asks the split to preserve roughly the same class proportions in training and test sets.

```
X_cls_train, X_cls_test, y_cls_train, y_cls_test = train_test_split(
    X_cls,
    y_cls,
    test_size=0.25,
    random_state=42,
    stratify=y_cls,
)
```

What to look for

This is especially useful for classification because a random split can otherwise distort class proportions.

Step 8.5: Import a classification model**Why this cell exists**

LogisticRegression is commonly used for binary classification. Despite the name, it is a classifier here.

```
from sklearn.linear_model import LogisticRegression
```

What to look for

This gives us the model class. It has not learned anything yet.

Step 8.6: Fit and predict classes**Why this cell exists**

The pattern is familiar: create the model, fit on training data, predict on test features.

```
classifier = LogisticRegression(max_iter=10000)
classifier.fit(X_cls_train, y_cls_train)

class_predictions = classifier.predict(X_cls_test)
class_predictions[:10]
```

What to look for

The predictions are class labels. They are not residuals or numeric regression predictions.

Step 8.7: Import classification metrics together**Why this cell exists**

These metrics all compare actual class labels with predicted class labels. They belong together, so this grouped import is small and purposeful, not an import-the-world cell.

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
```

```
recall_score,  
f1_score,  
confusion_matrix,  
)
```

What to look for

Accuracy is not enough. Precision, recall, F1, and the confusion matrix help reveal different kinds of wrongness.

Step 8.8: Evaluate the classifier

Why this cell exists

Use multiple metrics so you do not confuse overall correctness with catching the class that matters most.

```
print("Accuracy:", round(  
    accuracy_score(y_cls_test, class_predictions), 3  
))  
  
print("Precision:", round(  
    precision_score(y_cls_test, class_predictions), 3  
))  
  
print("Recall:", round(  
    recall_score(y_cls_test, class_predictions), 3  
))  
  
print("F1:", round(  
    f1_score(y_cls_test, class_predictions), 3  
))
```

What to look for

Do not memorise the numbers. Memorise the question: what kind of mistake would be expensive in this problem?

Step 8.9: Show the confusion matrix as a table

Why this cell exists

A confusion matrix counts actual-vs-predicted classes. It makes false positives and false negatives visible.

```
matrix = confusion_matrix(y_cls_test, class_predictions)  
  
pd.DataFrame(  
    matrix,  
    index=["actual_" + name for name in cancer.target_names],  
    columns=["predicted_" + name for name in cancer.target_names],  
)
```

What to look for

The diagonal cells are correct classifications. The off-diagonal cells are mistakes.